

Weather.keras

```
!pip install -q kaggle albumentations opencv-python ultralytics tensorflow scikit-learn matplotlib seaborn
print("All libraries installed.")
from google.colab import files
files.upload() # upload kaggle.json
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
!kaggle datasets download -d orville/dawn-detection-in-adverse-weather-nature
!unzip -q dawn-detection-in-adverse-weather-nature.zip -d dawn
```

```
!ls dawn/DAWN
print("DAWN downloaded.")
# Download BDD100K weather subset
!kaggle datasets download -d marquis03/bdd100k-weather-classification
!unzip -q bdd100k-weather-classification.zip -d bdd100k
!ls bdd100k
import os
```

```
bdd_train = "bdd100k/train"
bdd_val = "bdd100k/val"
```

```
print("BDD100K train classes:")
for cls in os.listdir(bdd_train):
    count = len(os.listdir(os.path.join(bdd_train, cls)))
    print(f" {cls}: {count} images")
```

```
print("\nBDD100K val classes:")
for cls in os.listdir(bdd_val):
    count = len(os.listdir(os.path.join(bdd_val, cls)))
    print(f" {cls}: {count} images")
!kaggle datasets download -d khalilusmanuk/acdc-clean-dataset-for-training
!unzip -q acdc-clean-dataset-for-training.zip -d acdc
!ls acdc
import os
```

```
# Walk entire acdc directory and find all image files
all_images = []
```

```

for root, dirs, files in os.walk("acdc"):
    for f in files:
        if f.lower().endswith(('.jpg', '.png', '.jpeg')):
            all_images.append(os.path.join(root, f))

print(f"Total images found: {len(all_images)}")
print("\nSample paths:")
for img in all_images[:10]:
    print(img)
import os

weather_counts = {'fog': [], 'rain': [], 'snow': []}

for root, dirs, files in os.walk("acdc"):
    for f in files:
        if f.lower().endswith(('.jpg', '.png', '.jpeg')):
            full_path = os.path.join(root, f)
            for weather in weather_counts:
                if f"/images/{weather}/" in full_path:
                    weather_counts[weather].append(full_path)

for weather, imgs in weather_counts.items():
    print(f"{weather}: {len(imgs)} images")
import os
import shutil
import random

random.seed(42)

base_path = "dawn/DAWN"

os.rename(f"{base_path}/Fog", f"{base_path}/fog")
os.rename(f"{base_path}/Rain", f"{base_path}/rain")
os.rename(f"{base_path}/Sand", f"{base_path}/sand")
os.rename(f"{base_path}/Snow", f"{base_path}/snow")

# — Collect ACDC fog images —————
acdc_fog = []
acdc_rain = []

for root, dirs, files in os.walk("acdc"):
    for f in files:
        if f.lower().endswith(('.jpg', '.png', '.jpeg')):

```

```

    full_path = os.path.join(root, f)
    if "/images/fog/" in full_path:
        acdc_fog.append(full_path)
    elif "/images/rain/" in full_path:
        acdc_rain.append(full_path)

# — Copy all ACDC fog → dawn/DAWN/fog —————
print(f"Copying {len(acdc_fog)} ACDC fog images...")
for img_path in acdc_fog:
    fname = os.path.basename(img_path)
    shutil.copy(img_path, os.path.join(base_path, "fog", f"acdc_{fname}"))

# — Copy all ACDC rain → dawn/DAWN/rain —————
print(f"Copying {len(acdc_rain)} ACDC rain images...")
for img_path in acdc_rain:
    fname = os.path.basename(img_path)
    shutil.copy(img_path, os.path.join(base_path, "rain", f"acdc_{fname}"))

# — Also add BDD100K rainy (500 images) —————
bdd_rainy_dir = "bdd100k/train/rainy"
rainy_imgs = os.listdir(bdd_rainy_dir)
random.shuffle(rainy_imgs)
rainy_subset = rainy_imgs[:500]
print(f"Copying {len(rainy_subset)} BDD100K rainy images...")
for img in rainy_subset:
    shutil.copy(
        os.path.join(bdd_rainy_dir, img),
        os.path.join(base_path, "rain", f"bdd_{img}")
    )

# — Verify final counts —————
print("\nFinal counts after merge:")
for cls in ['fog', 'rain', 'sand', 'snow']:
    count = 0
    for root, dirs, files in os.walk(f"{base_path}/{cls}"):
        count += sum(1 for f in files if f.lower().endswith(('.jpg', '.png', '.jpeg')))
    print(f" {cls}: {count} images")
import os
import shutil
import random

```

```

random.seed(42)

input_dir = "dawn/DAWN"
output_dir = "weather_dataset"
classes = ['fog', 'rain', 'sand', 'snow']

for cls in classes:
    os.makedirs(f"{output_dir}/train/{cls}", exist_ok=True)
    os.makedirs(f"{output_dir}/val/{cls}", exist_ok=True)

all_images = []
for root, dirs, files in os.walk(os.path.join(input_dir, cls)):
    for file in files:
        if file.lower().endswith(('.jpg', '.png', '.jpeg')):
            all_images.append(os.path.join(root, file))

random.shuffle(all_images)
split = int(0.8 * len(all_images))
train_imgs = all_images[:split]
val_imgs = all_images[split:]

for img_path in train_imgs:
    shutil.copy(img_path, os.path.join(output_dir, "train", cls))
for img_path in val_imgs:
    shutil.copy(img_path, os.path.join(output_dir, "val", cls))

print(f"{cls}: train={len(train_imgs)}, val={len(val_imgs)}")
import albumentations as A
import cv2
import os
import shutil
import random
import numpy as np

# — Fog: bright, uniform, soft haze —————
fog_aug = A.Compose([
    A.RandomFog(fog_coef_range=(0.4, 0.8), alpha_coef=0.1, p=1.0),
    A.RandomBrightnessContrast(brightness_limit=(0.1, 0.3), contrast_limit=(-0.3, -0.1), p=0.9),
    A.GaussianBlur(blur_limit=(5, 9), p=0.8),

```

```
A.RandomGamma(gamma_limit=(90, 130), p=0.5),
A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=(-40, -20), val_shift_limit=(10, 30), p=0.7),
])
```

```
# — Rain: dark, directional, blue-grey wet shift —————
```

```
rain_aug = A.Compose([
    A.RandomRain(slant_range=(-15, 15), drop_length=25, drop_width=1, blur_value=4,
    brightness_coefficient=0.75, p=1.0),
    A.MotionBlur(blur_limit=(5, 9), p=0.8),
    A.RandomBrightnessContrast(brightness_limit=(-0.3, -0.1), contrast_limit=(-0.1, 0.1), p=0.8),
    A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=(-20, 10), val_shift_limit=(-20, -5), p=0.7),
    A.RGBShift(r_shift_limit=(-10, 0), g_shift_limit=(-5, 5), b_shift_limit=(5, 20), p=0.7),
])
```

```
# — Snow: bright white, low saturation —————
```

```
snow_aug = A.Compose([
    A.RandomSnow(brightness_coeff=1.8, p=0.9),
    A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.05, p=0.5),
    A.GaussianBlur(blur_limit=(3, 5), p=0.4),
    A.HueSaturationValue(hue_shift_limit=5, sat_shift_limit=-20, val_shift_limit=15, p=0.4),
])
```

```
# — Sand: orange-brown tint, dusty haze —————
```

```
sand_aug = A.Compose([
    A.RandomFog(fog_coef_range=(0.2, 0.5), p=0.6),
    A.RGBShift(r_shift_limit=25, g_shift_limit=12, b_shift_limit=5, p=0.8),
    A.GaussNoise(std_range=(0.02, 0.08), p=0.6),
    A.GaussianBlur(blur_limit=(3, 7), p=0.6),
    A.RandomBrightnessContrast(brightness_limit=(-0.2, 0.1), contrast_limit=(-0.3, 0.1), p=0.6),
    A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.3, hue=0.05, p=0.5),
])
```

```
aug_map = {
    'fog': fog_aug,
    'rain': rain_aug,
    'snow': snow_aug,
    'sand': sand_aug,
}
```

```
# — Target per class —————
```

```
TARGET = {  
    'fog': 1500,  
    'rain': 1500,  
    'snow': 1000,  
    'sand': 1000,  
}
```

```
train_dir = "weather_dataset/train"  
final_dir = "final_dataset/train"
```

```
for cls in classes:
```

```
    out_dir = os.path.join(final_dir, cls)  
    os.makedirs(out_dir, exist_ok=True)
```

```
    src_dir = os.path.join(train_dir, cls)  
    src_imgs = os.listdir(src_dir)
```

```
    # Copy originals first
```

```
    for img_name in src_imgs:  
        shutil.copy(os.path.join(src_dir, img_name), os.path.join(out_dir, img_name))
```

```
    current = len(src_imgs)  
    target = TARGET[cls]  
    needed = max(0, target - current)  
    count = 0
```

```
    aug = aug_map[cls]
```

```
    while count < needed:
```

```
        img_name = random.choice(src_imgs)  
        img_cv = cv2.imread(os.path.join(src_dir, img_name))  
        if img_cv is None:  
            continue  
        img_cv = cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB)  
        augmented = aug(image=img_cv)['image']  
        cv2.imwrite(  
            os.path.join(out_dir, f"aug_{count}_{img_name}"),  
            cv2.cvtColor(augmented, cv2.COLOR_RGB2BGR)  
        )  
        count += 1
```

```

total = len(os.listdir(out_dir))
print(f"{cls}: {total} images (target={target})")
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.efficientnet import preprocess_input
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

train_dir_final = "final_dataset/train"
val_dir        = "weather_dataset/val"

train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=15,
    zoom_range=0.2,
    horizontal_flip=True,
    width_shift_range=0.1,
    height_shift_range=0.1,
)

val_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)

train_data = train_datagen.flow_from_directory(
    train_dir_final,
    target_size=(300, 300),
    batch_size=32,
    class_mode='categorical'
)

val_data = val_datagen.flow_from_directory(
    val_dir,
    target_size=(300, 300),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)

train_steps = len(train_data)

```

```

val_steps = len(val_data)

# Class weights
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_data.classes),
    y=train_data.classes
)
class_weights = dict(enumerate(class_weights))
cw_tensor = np.array([class_weights[i] for i in range(4)], dtype=np.float32)

def add_validity_labels(generator):
    for X, y_weather in generator:
        batch_size = X.shape[0]
        y_validity = np.ones((batch_size, 1), dtype=np.float32)
        class_indices_batch = np.argmax(y_weather, axis=1)
        sample_weights = cw_tensor[class_indices_batch]
        yield X, {'weather': y_weather, 'validity': y_validity}, sample_weights

print("Class indices:", train_data.class_indices)
print(f"Train batches: {train_steps}, Val batches: {val_steps}")
print("Class weights:", class_weights)
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras import layers, models

num_classes = len(train_data.class_indices)

base_model = EfficientNetB3(
    weights='imagenet',
    include_top=False,
    input_shape=(300, 300, 3)
)
base_model.trainable = False

x = base_model.output
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dense(512, activation='relu')(x)
x = layers.Dropout(0.4)(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.3)(x)

# Weather classification head
weather_output = layers.Dense(num_classes, activation='softmax', name='weather')(x)

```

```

# Validity head
validity_output = layers.Dense(1, activation='sigmoid', name='validity')(x)

model = models.Model(
    inputs=base_model.input,
    outputs=[weather_output, validity_output]
)

print(f"Model inputs : {model.input_shape}")
print(f"Total params : {model.count_params();}")
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau

callbacks = [
    ModelCheckpoint(
        "best_weather_model.keras",
        monitor='val_weather_accuracy',
        save_best_only=True,
        mode='max',
        verbose=1
    ),
    EarlyStopping(
        monitor='val_weather_accuracy',
        patience=6,
        mode='max',
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_weather_loss',
        factor=0.5,
        patience=3,
        mode='min',
        min_lr=1e-7,
        verbose=1
    )
]

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss={
        'weather': tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
        'validity': tf.keras.losses.BinaryCrossentropy()
    },
    loss_weights={
        'weather': 1.0,
        'validity': 0.3
    },
    metrics={
        'weather': 'accuracy',
        'validity': 'accuracy'
    }
)

```

```

train_data.reset()
val_data.reset()
train_gen = add_validity_labels(train_data)
val_gen = add_validity_labels(val_data)

print("Phase 1: Training top layers (base frozen)...")
history = model.fit(
    train_gen,
    steps_per_epoch=train_steps,
    validation_data=val_gen,
    validation_steps=val_steps,
    epochs=30,
    callbacks=callbacks
)
from tensorflow.keras.models import load_model

model = load_model("best_weather_model.keras")
print("Best Phase 1 model loaded.")

# Unfreeze all layers
for layer in model.layers:
    layer.trainable = True

# Freeze first 60 layers
for layer in model.layers[:60]:
    layer.trainable = False

trainable = sum(1 for l in model.layers if l.trainable)
print(f"Trainable layers: {trainable}")

callbacks_ft = [
    ModelCheckpoint(
        "best_weather_model_ft.keras",
        monitor='val_weather_accuracy',
        save_best_only=True,
        mode='max',
        verbose=1
    ),
    EarlyStopping(
        monitor='val_weather_accuracy',
        patience=8,
        mode='max',
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_weather_loss',
        factor=0.5,
        patience=3,
        mode='min',
        min_lr=1e-7,

```

```

        verbose=1
    )
]

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss={
        'weather': tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.05),
        'validity': tf.keras.losses.BinaryCrossentropy()
    },
    loss_weights={
        'weather': 1.0,
        'validity': 0.3
    },
    metrics={
        'weather': 'accuracy',
        'validity': 'accuracy'
    }
)

```

```

train_data.reset()
val_data.reset()
train_gen = add_validity_labels(train_data)
val_gen = add_validity_labels(val_data)

```

```

print("Phase 2: Fine-tuning...")
history_ft = model.fit(
    train_gen,
    steps_per_epoch=train_steps,
    validation_data=val_gen,
    validation_steps=val_steps,
    epochs=25,
    callbacks=callbacks_ft
)

```

```

from tensorflow.keras.models import load_model
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

```

```

model = load_model("best_weather_model_ft.keras")
print("Best model loaded.")

```

```

classes_list = ['fog', 'rain', 'sand', 'snow']

```

```

val_data.reset()
all_preds = []
all_labels = []

```

```

for i, (X, y_weather) in enumerate(val_data):
    preds = model.predict(X, verbose=0)
    weather_preds = preds[0]
    all_preds.extend(np.argmax(weather_preds, axis=1))
    all_labels.extend(np.argmax(y_weather, axis=1))
    if i >= val_steps - 1:
        break

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Confusion matrix
cm = confusion_matrix(all_labels, all_preds)
plt.figure(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes_list, yticklabels=classes_list)
plt.title('Confusion Matrix — Weather Classification')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()

print("\n--- Per-Class Report ---")
print(classification_report(all_labels, all_preds, target_names=classes_list))
import json
from google.colab import files

# Save final model
model.save("weather_final.keras")
print("Model saved as weather_final.keras")

# Save class indices
class_indices = train_data.class_indices
with open("class_indices.json", "w") as f:
    json.dump(class_indices, f)
print("Class indices saved:", class_indices)

# Save confusion matrix

```

```
print("Confusion matrix saved as confusion_matrix.png")
```

```
# Download everything immediately
files.download("weather_final.keras")
files.download("class_indices.json")
files.download("confusion_matrix.png")
```

app.py

```
from flask import Flask, request, render_template
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.efficientnet import preprocess_input
from ultralytics import YOLO

import numpy as np
import os
import cv2
import json
import tensorflow as tf
```

```
app = Flask(__name__)
```

```
# — Load models —————  
weather_model = load_model("weather_final.keras")  
yolo_model = YOLO("yolov8s.pt")
```

```
# — Class labels —————  
with open("class_indices.json") as f:  
    class_indices = json.load(f)  
classes = [k for k, v in sorted(class_indices.items(), key=lambda x: x[1])]
print("Classes:", classes)
```

— Road-relevant YOLO object classes —————

```
PRIMARY_ROAD_CLASSES = {  
    'car', 'truck', 'bus', 'motorcycle', 'bicycle',  
    'traffic light', 'stop sign', 'train', 'van'  
}
```

```
SECONDARY_ROAD_CLASSES = {'person'}
```

— Weather-specific YOLO confidence thresholds —————

```
WEATHER_CONF = {  
    'fog': 0.25,  
    'rain': 0.30,  
    'sand': 0.25,  
    'snow': 0.35,  
}
```

— Validity thresholds —————

```
WEATHER_CONF_THRESHOLD = 0.60  
MIN_TOP2_GAP           = 0.15
```

```
UPLOAD_FOLDER = "static/uploads"  
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
```

— TTA prediction —————

```
def predict_weather_tta(filepath, n_augments=10):  
    img     = image.load_img(filepath, target_size=(300, 300))  
    img_array = image.img_to_array(img)
```

```
tta_datagen = tf.keras.preprocessing.image.ImageDataGenerator(  
    preprocessing_function=preprocess_input,  
    horizontal_flip=True,  
    zoom_range=0.1,  
    brightness_range=[0.85, 1.15],  
    fill_mode='nearest'  
)
```

```
preds = []
```

```
validity = []
```

```
# Original pass
```

```
orig = preprocess_input(img_array.copy())
```

```
outputs = weather_model.predict(np.expand_dims(orig, 0), verbose=0)
```

```
preds.append(outputs[0][0])
```

```
validity.append(float(outputs[1][0][0]))
```

```
# Augmented passes
```

```
for x_aug in tta_datagen.flow(np.expand_dims(img_array, 0), batch_size=1):
```

```
    outputs = weather_model.predict(x_aug, verbose=0)
```

```
    preds.append(outputs[0][0])
```

```
    validity.append(float(outputs[1][0][0]))
```

```
    if len(preds) >= n_augments:
```

```
        break
```

```
avg_pred = np.mean(preds, axis=0)
```

```
avg_validity = np.mean(validity)
```

```
weather = classes[np.argmax(avg_pred)]
```

```
confidence = float(np.max(avg_pred)) * 100
```

```
return weather, confidence, avg_pred, avg_validity
```

```
# — Invalid input checker —————  
  
def is_invalid_input(weather_confidence, all_probs, detected_objects):  
    detected_lower = [obj.lower() for obj in detected_objects]  
  
    primary_found = any(obj in PRIMARY_ROAD_CLASSES for obj in detected_lower)  
    secondary_found = any(obj in SECONDARY_ROAD_CLASSES for obj in detected_lower)  
  
    # Top prediction must clearly dominate 2nd prediction  
    sorted_probs = sorted(all_probs, reverse=True)  
    top1_prob = sorted_probs[0]  
    top2_prob = sorted_probs[1]  
    gap = top1_prob - top2_prob  
  
    confidence_weak = weather_confidence < WEATHER_CONF_THRESHOLD  
    prediction_unclear = gap < MIN_TOP2_GAP  
  
    # No primary road object detected  
    if not primary_found:  
        if secondary_found:  
            # Person detected but no vehicle  
            # Invalid if confidence weak OR prediction unclear  
            if confidence_weak or prediction_unclear:  
                return True  
        else:  
            # Nothing road-relevant at all → always invalid  
            return True
```

```
return False
```

```
@app.route('/', methods=['GET', 'POST'])
```

```
def index():
```

```
    if request.method == 'POST':
```

```
        # — Get uploaded file —————
```

```
        file = request.files['image']
```

```
        filepath = os.path.join(UPLOAD_FOLDER, file.filename)
```

```
        file.save(filepath)
```

```
        # — Weather prediction with TTA —————
```

```
        weather, confidence, all_probs, avg_validity = predict_weather_tta(filepath)
```

```
        prob_dict = {c: f"{p*100:.1f}%" for c, p in zip(classes, all_probs)}
```

```
        # — YOLO detection with weather-specific threshold ———
```

```
        conf_threshold = WEATHER_CONF.get(weather, 0.35)
```

```
        results = yolo_model(filepath, conf=conf_threshold)
```

```
        detected = []
```

```
        for r in results:
```

```
            for box in r.bboxes:
```

```
                cls_id = int(box.cls[0])
```

```
                detected.append(yolo_model.names[cls_id])
```

```
        detected = list(set(detected))
```

```
        # — Invalid input check —————
```

```
        if is_invalid_input(confidence, all_probs, detected):
```

```
            return render_template(
```

```
    "index.html",
    error="Invalid input — please upload a road or weather scene image.",
    image=filepath
)
```

```
# — Draw bounding boxes —————
```

```
img_with_boxes = None
```

```
for r in results:
```

```
    img_with_boxes = r.plot()
```

```
output_path = os.path.join(UPLOAD_FOLDER, "result_" + file.filename)
```

```
if img_with_boxes is not None:
```

```
    cv2.imwrite(output_path, img_with_boxes)
```

```
else:
```

```
    output_path = filepath
```

```
# — Render result —————
```

```
return render_template(
```

```
    "index.html",
```

```
    weather=weather,
```

```
    confidence=f"{confidence:.1f}%",
```

```
    prob_dict=prob_dict,
```

```
    objects=detected,
```

```
    image=output_path,
```

```
    validity_score=f"{avg_validity:.2f}"
```

```
)
```

```
return render_template("index.html")
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

index.html

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  
    <meta charset="UTF-8"/>  
  
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>  
  
    <title>StormEye — Weather & Road Detection</title>  
  
    <link  
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800&family=Crim  
son+Pro:ital,wght@0,300;0,400;1,300&display=swap" rel="stylesheet"/>  
  
    <style>  
  
        *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }  
  
  
        :root {  
  
            --bg:    #f0f4ff;  
  
            --bg2:   #e6ecfa;  
  
            --surface: #ffffff;  
  
            --surface2: #f7f9ff;  
  
            --border:  rgba(99,120,200,0.13);  
  
            --border2: rgba(99,120,200,0.22);  
  
            --accent:  #3a6ef5;  
  
            --accent2: #00c49a;  
  
            --accent3: #7c4dff;  
  
            --text:    #1a1f36;  
  
            --muted:   #7a85a8;  
  
            --fog:     #5b9bd5;
```

```
--rain:    #3a6ef5;
--sand:    #d48c2e;
--snow:    #6eae0;
--shadow:  rgba(58,110,245,0.10);
--shadow2: rgba(58,110,245,0.18);
}
```

```
body {
  font-family: 'Outfit', sans-serif;
  background: var(--bg);
  color: var(--text);
  min-height: 100vh;
  overflow-x: hidden;
}
```

```
body::before {
  content: "";
  position: fixed;
  top: -180px; right: -180px;
  width: 520px; height: 520px;
  background: radial-gradient(circle, rgba(124,77,255,0.10) 0%, transparent 70%);
  pointer-events: none;
  z-index: 0;
}
```

```
body::after {
  content: "";
  position: fixed;
  bottom: -160px; left: -160px;
  width: 480px; height: 480px;
```

```
background: radial-gradient(circle, rgba(58,110,245,0.09) 0%, transparent 70%);
pointer-events: none;
z-index: 0;
}
```

```
.wrapper {
  position: relative;
  z-index: 1;
  max-width: 880px;
  margin: 0 auto;
  padding: 52px 24px 90px;
}
```

```
header {
  margin-bottom: 44px;
  animation: fadeDown .55s ease both;
}
```

```
.logo-row {
  display: flex;
  align-items: center;
  gap: 14px;
  margin-bottom: 10px;
}
```

```
.logo-mark {
  width: 48px; height: 48px;
  background: linear-gradient(135deg, var(--accent), var(--accent3));
  border-radius: 14px;
  display: flex; align-items: center; justify-content: center;
```

```
font-size: 24px;
box-shadow: 0 6px 20px var(--shadow2);
flex-shrink: 0;
}
```

```
.brand {
font-size: clamp(28px, 5vw, 42px);
font-weight: 800;
letter-spacing: -0.03em;
color: var(--text);
line-height: 1;
}
```

```
.brand span {
background: linear-gradient(90deg, var(--accent), var(--accent3));
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
background-clip: text;
}
```

```
.tagline {
font-family: 'Crimson Pro', serif;
font-style: italic;
font-size: 17px;
color: var(--muted);
margin-top: 4px;
font-weight: 300;
}
```

```
.divider {
```

```
width: 56px; height: 3px;
background: linear-gradient(90deg, var(--accent), var(--accent3));
border-radius: 99px;
margin: 18px 0 0;
}
```

```
.upload-card {
background: var(--surface);
border: 1px solid var(--border);
border-radius: 22px;
padding: 36px;
margin-bottom: 24px;
box-shadow: 0 4px 32px var(--shadow);
animation: fadeUp .55s .08s ease both;
position: relative;
overflow: hidden;
}
```

```
.upload-card::before {
content: "";
position: absolute;
top: 0; left: 0; right: 0;
height: 3px;
background: linear-gradient(90deg, var(--accent), var(--accent3), var(--accent2));
}
```

```
.section-label {
font-size: 11px;
font-weight: 600;
letter-spacing: .12em;
```

```
text-transform: uppercase;
color: var(--muted);
margin-bottom: 18px;
display: flex;
align-items: center;
gap: 8px;
}
```

```
.section-label::after {
  content: ";
  flex: 1;
  height: 1px;
  background: var(--border2);
}
```

```
.drop-zone {
  border: 1.5px dashed rgba(58,110,245,0.28);
  border-radius: 16px;
  padding: 40px 24px;
  text-align: center;
  cursor: pointer;
  transition: all .2s;
  margin-bottom: 20px;
  position: relative;
  background: var(--surface2);
}
```

```
.drop-zone:hover {
  border-color: var(--accent);
  background: rgba(58,110,245,0.04);
}
```

```
    box-shadow: 0 0 0 4px rgba(58,110,245,0.06);  
}
```

```
.drop-zone input[type=file] {  
    position: absolute;  
    inset: 0;  
    opacity: 0;  
    cursor: pointer;  
    width: 100%;  
    height: 100%;  
}
```

```
.drop-icon { font-size: 36px; display: block; margin-bottom: 12px; }
```

```
.drop-title {  
    font-size: 16px;  
    font-weight: 500;  
    color: var(--text);  
    margin-bottom: 4px;  
}
```

```
.drop-sub { font-size: 13px; color: var(--muted); }  
.drop-sub span { color: var(--accent); font-weight: 500; }
```

```
.file-name {  
    font-size: 13px;  
    color: var(--accent2);  
    font-weight: 500;  
    margin-top: 10px;  
    min-height: 18px;
```

```
}
```

```
.btn {  
  display: inline-flex;  
  align-items: center;  
  gap: 8px;  
  background: linear-gradient(135deg, var(--accent), var(--accent3));  
  color: #fff;  
  font-family: 'Outfit', sans-serif;  
  font-size: 15px;  
  font-weight: 600;  
  padding: 13px 36px;  
  border: none;  
  border-radius: 12px;  
  cursor: pointer;  
  box-shadow: 0 4px 18px rgba(58,110,245,0.30);  
  transition: transform .15s, box-shadow .15s;  
  letter-spacing: .01em;  
}
```

```
.btn:hover { transform: translateY(-2px); box-shadow: 0 8px 24px rgba(58,110,245,0.38); }
```

```
.btn:active { transform: translateY(0); }
```

```
.error-card {  
  background: #fff5f5;  
  border: 1px solid rgba(229,62,62,0.2);  
  border-radius: 16px;  
  padding: 20px 24px;  
  display: flex;  
  align-items: flex-start;
```

```
gap: 14px;
margin-bottom: 20px;
animation: fadeUp .4s ease both;
box-shadow: 0 2px 12px rgba(229,62,62,0.07);
}
```

```
.error-icon { font-size: 22px; flex-shrink: 0; }
```

```
.error-text {
font-size: 15px;
color: #c53030;
line-height: 1.5;
font-weight: 500;
}
```

```
.error-sub {
font-size: 13px;
color: #e05252;
margin-top: 4px;
font-weight: 400;
}
```

```
.result-grid {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 16px;
margin-bottom: 16px;
animation: fadeUp .5s .05s ease both;
}
```

```
@media (max-width: 560px) { .result-grid { grid-template-columns: 1fr; } }
```

```
.result-card {  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: 18px;  
  padding: 26px;  
  box-shadow: 0 4px 20px var(--shadow);  
  position: relative;  
  overflow: hidden;  
}
```

```
.weather-icon-bg {  
  position: absolute;  
  right: -10px; top: -10px;  
  font-size: 80px;  
  opacity: 0.07;  
  pointer-events: none;  
  line-height: 1;  
}
```

```
.weather-name {  
  font-size: 36px;  
  font-weight: 800;  
  text-transform: capitalize;  
  letter-spacing: -0.02em;  
  line-height: 1;  
  margin: 10px 0 6px;  
}
```

```
.w-fog { color: var(--fog); }  
.w-rain { color: var(--rain); }  
.w-sand { color: var(--sand); }  
.w-snow { color: var(--snow); }  
.w-default { color: var(--accent); }
```

```
.conf-pill {  
  display: inline-flex;  
  align-items: center;  
  gap: 5px;  
  background: rgba(0,196,154,0.10);  
  border: 1px solid rgba(0,196,154,0.22);  
  color: #00a07e;  
  font-size: 13px;  
  font-weight: 600;  
  padding: 4px 12px;  
  border-radius: 99px;  
}
```

```
.conf-dot {  
  width: 6px; height: 6px;  
  border-radius: 50%;  
  background: var(--accent2);  
  animation: pulse 2s ease infinite;  
}
```

```
.prob-list { display: flex; flex-direction: column; gap: 11px; }
```

```
.prob-row { display: flex; align-items: center; gap: 10px; }
```

```
.prob-name {  
  font-size: 12px;  
  font-weight: 600;  
  text-transform: capitalize;  
  color: var(--muted);  
  width: 38px;  
  flex-shrink: 0;  
}
```

```
.prob-track {  
  flex: 1;  
  height: 7px;  
  background: var(--bg2);  
  border-radius: 99px;  
  overflow: hidden;  
}
```

```
.prob-fill {  
  height: 100%;  
  border-radius: 99px;  
  transition: width .7s cubic-bezier(.4,0,.2,1);  
}
```

```
.fill-fog { background: linear-gradient(90deg, var(--fog), #90c4e8); }  
.fill-rain { background: linear-gradient(90deg, var(--rain), #6f9ef8); }  
.fill-sand { background: linear-gradient(90deg, var(--sand), #e8b96a); }  
.fill-snow { background: linear-gradient(90deg, var(--snow), #a8d4f0); }
```

```
.prob-pct {  
  font-size: 12px;
```

```
color: var(--text);  
font-weight: 600;  
width: 38px;  
text-align: right;  
flex-shrink: 0;  
}
```

```
.obj-wrap {  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: 18px;  
  padding: 24px 26px;  
  margin-bottom: 16px;  
  box-shadow: 0 4px 20px var(--shadow);  
  animation: fadeUp .5s .1s ease both;  
}
```

```
.obj-tags {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 8px;  
  margin-top: 14px;  
}
```

```
.obj-tag {  
  background: linear-gradient(135deg, rgba(58,110,245,0.08), rgba(124,77,255,0.08));  
  border: 1px solid rgba(58,110,245,0.16);  
  color: var(--accent);  
  padding: 6px 16px;  
  border-radius: 99px;
```

```
font-size: 13px;  
font-weight: 600;  
}
```

```
.no-objects {  
  color: var(--muted);  
  font-size: 14px;  
  margin-top: 12px;  
  font-style: italic;  
}
```

```
.img-wrap {  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: 18px;  
  overflow: hidden;  
  box-shadow: 0 4px 24px var(--shadow);  
  animation: fadeUp .5s .15s ease both;  
}
```

```
.img-header {  
  padding: 14px 22px;  
  border-bottom: 1px solid var(--border);  
  display: flex;  
  align-items: center;  
  gap: 10px;  
}
```

```
.live-badge {  
  display: flex;
```

```
align-items: center;
gap: 6px;
background: rgba(0,196,154,0.10);
border: 1px solid rgba(0,196,154,0.2);
color: #00a07e;
font-size: 11px;
font-weight: 700;
letter-spacing: .08em;
text-transform: uppercase;
padding: 3px 10px;
border-radius: 99px;
}
```

```
.live-dot {
width: 6px; height: 6px;
border-radius: 50%;
background: var(--accent2);
animation: pulse 1.5s ease infinite;
}
```

```
.img-title {
font-size: 13px;
font-weight: 600;
color: var(--muted);
letter-spacing: .04em;
}
```

```
.img-wrap img { width: 100%; display: block; }
```

```
.error-img-wrap {
```

```
background: var(--surface);
border: 1px solid var(--border);
border-radius: 16px;
overflow: hidden;
margin-top: 16px;
box-shadow: 0 2px 14px var(--shadow);
}
```

```
.error-img-wrap img { width: 100%; display: block; }
```

```
footer {
  margin-top: 52px;
  text-align: center;
  font-size: 13px;
  color: var(--muted);
  font-family: 'Crimson Pro', serif;
  font-style: italic;
}
```

```
@keyframes fadeDown {
  from { opacity: 0; transform: translateY(-18px); }
  to   { opacity: 1; transform: translateY(0); }
}
```

```
@keyframes fadeUp {
  from { opacity: 0; transform: translateY(18px); }
  to   { opacity: 1; transform: translateY(0); }
}
```

```
@keyframes pulse {
```

```
    0%, 100% { opacity: 1; transform: scale(1); }
    50%     { opacity: .5; transform: scale(.85); }
  }
</style>
</head>
<body>
<div class="wrapper">

<header>
  <div class="logo-row">
    <div class="logo-mark">☐</div>
    <h1 class="brand">Weather<span>Lens</span></h1>
  </div>
  <p class="tagline">Adverse weather intelligence for autonomous driving systems</p>
  <div class="divider"></div>
</header>

<div class="upload-card">
  <p class="section-label">Upload Scene</p>
  <form method="POST" enctype="multipart/form-data">
    <div class="drop-zone" id="dropZone">
      <input type="file" name="image" accept="image/*" required id="fileInput"/>
      <span class="drop-icon">☐☐</span>
      <p class="drop-title">Drop your road scene image here</p>
      <p class="drop-sub">or <span>browse from device</span> · JPG, PNG, JPEG</p>
      <p class="file-name" id="fileName"></p>
    </div>
    <button type="submit" class="btn">☐&nbsp;Analyse Scene</button>
  </form>
</div>
```

```
{% if error %}
```

```
<div class="error-card">
```

```
  <span class="error-icon">❌</span>
```

```
  <div>
```

```
    <p class="error-text">{{ error }}</p>
```

```
    <p class="error-sub">Please upload a road or driving scene image for accurate detection.</p>
```

```
  </div>
```

```
</div>
```

```
{% if image %}
```

```
<div class="error-img-wrap">
```

```
  
```

```
</div>
```

```
{% endif %}
```

```
{% elif weather %}
```

```
{% set w = weather | lower %}
```

```
{% set icon = '☁️' if w == 'fog' else ('🌧️' if w == 'rain' else ('🏜️' if w == 'sand' else ('❄️' if w == 'snow' else '🌤️')) %}
```

```
<div class="result-grid">
```

```
  <div class="result-card">
```

```
    <span class="weather-icon-bg">{{ icon }}</span>
```

```
    <p class="section-label" style="margin-bottom:6px;">Detected Weather</p>
```

```
    <div class="weather-name w-{{ w if w in ['fog','rain','sand','snow'] else 'default' }}">
```

```
      {{ icon }} {{ weather }}
```

```
    </div>
```

```
  <div class="conf-pill" style="margin-top:10px;">
```

```
    <span class="conf-dot"></span>
```

```
    {{ confidence }} confidence
```

```
</div>
```

</div>

<div class="result-card">

<p class="section-label" style="margin-bottom:14px;">Class Probabilities</p>

{% if prob_dict %}

<div class="prob-list">

{% for cls, pct in prob_dict.items() %}

<div class="prob-row">

{{ cls }}

<div class="prob-track">

<div class="prob-fill fill-{{ cls }}"

style="width: {{ pct | replace('%','') | float }}%"></div>

</div>

{{ pct }}

</div>

{% endfor %}

</div>

{% endif %}

</div>

</div>

<div class="obj-wrap">

<p class="section-label">Detected Road Objects</p>

{% if objects %}

<div class="obj-tags">

{% for obj in objects %}

{{ obj }}

{% endfor %}

</div>

{% else %}

```

    <p class="no-objects">No road-relevant objects detected in this scene.</p>
    {% endif %}
</div>

{% if image %}
<div class="img-wrap">
    <div class="img-header">
        <span class="live-badge"><span class="live-dot"></span>Live</span>
        <span class="img-title">Detection Output — YOLOv8</span>
    </div>
    
</div>
{% endif %}

{% endif %}

<footer>StormEye · Adverse Weather Detection System · Autonomous Driving AI</footer>

</div>

<script>
const input  = document.getElementById('fileInput');
const fileName = document.getElementById('fileName');
const dropZone = document.getElementById('dropZone');

input.addEventListener('change', () => {
    if (input.files.length > 0)
        fileName.textContent = '✓ ' + input.files[0].name;
});

```

```
dropZone.addEventListener('dragover', e => {  
  e.preventDefault();  
  dropZone.style.borderColor = 'var(--accent)';  
  dropZone.style.background = 'rgba(58,110,245,0.05)';  
});
```

```
dropZone.addEventListener('dragleave', () => {  
  dropZone.style.borderColor = "";  
  dropZone.style.background = "";  
});
```

```
dropZone.addEventListener('drop', e => {  
  e.preventDefault();  
  dropZone.style.borderColor = "";  
  dropZone.style.background = "";  
  if (e.dataTransfer.files.length > 0) {  
    input.files = e.dataTransfer.files;  
    fileName.textContent = '✓ ' + e.dataTransfer.files[0].name;  
  }  
});
```

</script>

</body>

</html>